

Clase 12 – Next.js: Fundamentos y Rendering

IIC2182 – Interfaces y Experiencia de Usuario



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Primer Semestre 2026

Agenda

De React como librería a Next.js como framework.

Fundamentos

- ⚠️ Límites de React solo
- 🔧 ¿Qué es Next.js?
- 📁 Setup y estructura

Routing y Rendering

- 🏠 App Router (file-based)
- 🕒 Server vs Client Components
- SSG, SSR, ISR, CSR

Data Fetching

- 📡 Server-side fetch
- Cache y revalidate
- 🔗 SWR y TanStack Query



Hoy el cambio clave: **el servidor vuelve a ser protagonista** en el rendering de React.

Los límites de React solo

¿Por qué necesitamos más que una librería de UI?

React solo no alcanza

Construir una app real requiere resolver estos 4 problemas por tu cuenta.

1. SEO pobre

Una SPA sirve un HTML vacío + un bundle JS.
Los crawlers simples ven una página en blanco.
Productos, blogs, landing pages — SEO importa.

2. Routing manual

Hay que instalar react-router.
Configurar cada ruta a mano.
Sin convención → cada proyecto reinventa la rueda.

3. Sin server-side rendering

Todo el fetch ocurre después de cargar el JS.
Primera pantalla lenta (waterfall).
Mala experiencia en conexiones lentas.

4. Bundle inflado

Se envía todo el JS al cliente, aunque no lo use.
Sin code-splitting automático.
Sin optimización de imágenes, fuentes, scripts.

 Necesitamos un **framework** que resuelva esto con **convenciones**. Existen varios — Next.js, Remix, Gatsby — pero hoy nos centramos en **Next.js**, el más completo y usado del ecosistema.

¿Qué es Next.js?

“ Next.js

Un framework React creado por Vercel que agrega routing, rendering en servidor, optimización y convenciones para construir apps web de producción.

💡 La idea central

React es una librería para construir UI.

Next.js es un framework completo sobre React.

Sigue siendo React — los mismos componentes, hooks y JSX que ya conoces.

☆ ¿Por qué es relevante?

Usado por Netflix, TikTok, Hulu, Twitch, Nike, OpenAI.

Más de 125k stars en GitHub.

El "default" recomendado por el equipo de React.

Stack mental

Next.js — framework

corre sobre

React — librería de UI

compila a

HTML + CSS + JS

Los pilares de Next.js

Seis áreas donde Next.js aporta valor sobre React puro.



Routing

Basado en carpetas — cada archivo `page.tsx` es una ruta. Layouts anidados automáticos.



Rendering

SSG, SSR, ISR y CSR — elegís por ruta o por componente.



Data Fetching

fetch nativo en componentes servidor. Cache y revalidación automáticas.



Caching

Router cache, data cache, full route cache — cuatro capas trabajando juntas.



Optimization

`next/image`, `next/font`, code-splitting y prefetch automáticos.



TypeScript

Soporte zero-config, types para rutas y params, validación de links.

Setup y estructura

Crear un proyecto

```
npx create-next-app@latest nextjs-demo  
# --typescript --tailwind --app  
cd nextjs-demo  
npm run dev
```

Scripts principales

```
npm run dev — servidor de desarrollo  
npm run build — build de producción  
npm run start — ejecuta el build  
npm run lint — ESLint
```

Estructura de carpetas

```
nextjs-demo/  
├── app/  
│   ├── layout.tsx  
│   ├── page.tsx  
│   ├── globals.css  
│   └── about/  
│       └── page.tsx  
├── public/  
├── next.config.js  
├── tsconfig.json  
└── package.json
```

- ← App Router
- ← layout raíz (obligatorio)
- ← ruta /
- ← ruta /about
- ← assets estáticos

Cada carpeta dentro de `app/` es un segmento de URL. El archivo `page.tsx` la hace accesible.

App Router vs Pages Router

Next.js tiene dos sistemas de routing. Hoy nos centramos en el App Router, pero es útil saber que existe el otro.

Pages Router (legacy)

Carpeta `pages/`

Next.js ≤ 12 , introducido en 2016.

- `getServerSideProps`
- `getStaticProps`
- API Routes en `pages/api/`

Todavía soportado (código existente sigue funcionando) pero en modo mantenimiento. Next invierte en App Router.

App Router (moderno)

Carpeta `app/`

Next.js 13+ (estable desde Next 14, 2023).

- ✓ React Server Components
- ✓ Fetch nativo con cache integrado
- ✓ Layouts anidados, streaming, Server Actions

El default en `create-next-app` desde 2023. Es lo que usaremos.

 Si ves `pages/` en un proyecto viejo, es Pages Router. Si ves `app/`, es el moderno.

File-based Routing

Las carpetas son rutas. Los archivos son páginas.

page.tsx — una ruta

Cada carpeta dentro de `app/` es un segmento de URL. Un archivo `page.tsx` la hace accesible.

Estructura

```
app/
├── page.tsx           → /
├── about/
│   └── page.tsx      → /about
└── dashboard/
    ├── page.tsx      → /dashboard
    └── settings/
        └── page.tsx   → /dashboard/settings
```

Ejemplo mínimo

```
// app/about/page.tsx
export default function AboutPage() {
  return (
    <main>
      <h1>Sobre nosotros</h1>
      <p>IIC2182 – UX + Ingeniería Web</p>
    </main>
  )
}
```

Es un componente React normal. Sin imports especiales, sin registrar rutas.



La URL se deriva de la **estructura de carpetas**. No hay archivo de configuración de rutas.

layout.tsx — UI compartida

Un `layout.tsx` envuelve a sus hijos. Persiste entre navegaciones — no se re-renderiza.

```
// app/layout.tsx — layout raíz (obligatorio)
export default function RootLayout({
  children,
}): {
  children: React.ReactNode
}) {
  return (
    <html lang="es">
      <body>
        <nav>Mi App</nav>
        {children}
        <footer>© 2026</footer>
      </body>
    </html>
  )
}
```



Características

Obligatorio en la raíz de `app/`.

Recibe `children` como prop.

Persiste entre navegaciones.

Puede hacer fetch de datos (es server component).



Casos típicos

Navbar global

Footer

Providers (theme, auth, query client)

Fuentes globales y metadata

Layouts anidados

Cada carpeta puede tener su propio `layout.tsx`. Se acumulan de afuera hacia adentro.

```
app/
├── layout.tsx      ← Root (navbar + footer)
├── page.tsx
└── dashboard/
    ├── layout.tsx  ← Sidebar de dashboard
    ├── page.tsx
    └── settings/
        └── page.tsx ← Hereda root + dashboard
```

Para `/dashboard/settings` :

```
<RootLayout>           { /* navbar + footer */ }
  <DashboardLayout>     { /* sidebar */ }
    <SettingsPage />    { /* contenido */ }
  </DashboardLayout>
</RootLayout>
```

Al navegar entre rutas que comparten layout, el layout no se re-renderiza. Esto preserva scroll, state del sidebar, etc.



Layouts anidados = composición declarativa. Sin `React.Context` manual, sin `PageLayout` wrapper repetido.

Rutas dinámicas

Carpetas entre corchetes `param` capturan valores de la URL.

Estructura

```
app/  
└─ posts/  
   └─ page.tsx      → /posts  
   └─ [id]/  
      └─ page.tsx    → /posts/1, /posts/42
```

Catch-all

```
app/  
└─ docs/  
   └─ [... slug]/  
      └─ page.tsx    → /docs/a, /docs/a/b/c
```

Acceder al param

```
// app/posts/[id]/page.tsx  
export default async function PostPage({  
  params,  
}: {  
  params: Promise<{ id: string }>  
}) {  
  const { id } = await params  
  const post = await fetch(  
    `https://api.example.com/posts/${id}`  
  ).then(r => r.json())  
  
  return <article>{post.title}</article>  
}
```

En Next 15+, `params` es una Promise — se hace `await` .

Archivos especiales

Next.js reserva nombres de archivo para casos comunes — cada carpeta puede tenerlos.

loading.tsx

Se muestra mientras el segmento carga.
Usa `Suspense` por debajo — streaming UI.
Skeleton, spinner, shimmer.

error.tsx

Error boundary del segmento.
Recibe `error` y `reset()`.
Debe ser client component.

not-found.tsx

Página 404 personalizada.
Dispara con `notFound()`.
También captura URLs no existentes.

route.ts

Endpoint HTTP — no es una página.
Exporta `GET`, `POST`, etc.
Lo vemos en clase 13.



Estos archivos son **convención, no configuración**. Los pones en la carpeta y Next los usa automáticamente.

Navegación: <Link>

```
import Link from 'next/link'

export default function Nav() {
  return (
    <nav>
      <Link href="/">Inicio</Link>
      <Link href="/about">Sobre</Link>
      <Link href="/posts/42">Post 42</Link>
    </nav>
  )
}
```

 ¿Por qué no <a>?

<a> recarga la página completa.

<Link> hace client-side navigation.

El layout no se re-renderiza. Es instantáneo.

 Prefetch automático

Next precarga las rutas de los Links visibles.

Cuando el usuario hace clic, ya tiene los datos.

Configurable con `prefetch={false}` .



Regla: **siempre** <Link> para navegación interna. Usa <a> solo para URLs externas.

Server vs Client Components

El cambio de paradigma más importante de Next moderno

El cambio de paradigma

← Antes: React como SPA

Todo se ejecuta en el navegador.

El servidor solo sirve HTML vacío + JS.

Cada componente es "Client Component".

Para hacer fetch de un post: JS llega → ejecuta → useEffect → fetch → re-render.

→ Ahora: Server first

Por default, los componentes son Server Components.

Se ejecutan en el servidor y envían HTML listo.

Solo los que necesitan interactividad se marcan como "Client".

Para hacer fetch de un post: el servidor ya lo trae y envía HTML. Cero JS en el cliente.

💡 En el App Router, **Server es el default**. Pensá "¿realmente necesito JS en el cliente?" antes de agregar `"use client"`.

Server Component

```
// app/posts/page.tsx
// (sin "use client" - es server por default)

export default async function PostsPage() {
  const posts = await fetch(
    'https://api.example.com/posts'
  ).then(r => r.json())

  return (
    <ul>
      {posts.map(p => (
        <li key={p.id}>{p.title}</li>
      ))}
    </ul>
  )
}
```

✓ Qué puede hacer

- Fetch directo con `async/await`
- Leer archivos, acceder a bases de datos
- Usar secrets / variables de entorno
- Renderizar a HTML en el servidor

✗ Qué NO puede hacer

- `useState` , `useEffect` , hooks
- `onClick` , event handlers
- APIs del browser (`window` , `localStorage`)

⚡ El beneficio

Cero JS enviado al cliente para este componente.

Client Component

```
'use client'  
  
import { useState } from 'react'  
  
export default function Counter() {  
  const [count, setCount] = useState(0)  
  
  return (  
    <button onClick={() => setCount(c => c + 1)}>  
      Clicks: {count}  
    </button>  
  )  
}
```

La directiva "use client" va en la primera línea del archivo.

✓ Qué puede hacer

Todos los hooks: `useState` , `useEffect` , etc.

Event handlers (`onClick` , `onChange`)

Browser APIs (`window` , `localStorage`)

Librerías client-only (Zustand, Framer Motion)

⚠ El costo

Se hidrata en el cliente.

Envía JavaScript al bundle.

Más peso = carga inicial más lenta.

💡 Dato importante

Los Client Components también se renderizan en el servidor (para HTML inicial) — pero también en el cliente.

¿Cuál uso? — Árbol de decisión

¿Este componente necesita...?

👤 Interactividad (`useState` , `clicks`, `forms`)

🔄 Efectos (`useEffect` , `polling`)

APIs del browser (`localStorage`, `window`, `geolocalización`)

📦 Librerías client-only (`Zustand`, `Framer`)

→ Client Component (`"use client"`)

🌐 Fetch de datos en el servidor

🗄 Acceso a DB / secrets

📄 Display estático (texto, imágenes)

🏠 Layout / estructura

→ Server Component (default)



Default: **Server**. Agrega `"use client"` solo cuando realmente lo necesites.

Patrón: Client como hojas

Mantené los Client Components pequeños y en las hojas del árbol.

❌ Mal patrón

Marcar el layout raíz como "use client" .

Todo el árbol se vuelve client → pierde los beneficios del server.

```
// ❌ app/layout.tsx
'use client'

export default function RootLayout() {
  // Todo el árbol se hidrata
}
```

✅ Buen patrón

Layout y páginas: Server.

Botones, forms, widgets interactivos: Client.

```
// ✅ app/page.tsx (Server)
import SearchBar from './SearchBar'
// ↑ SearchBar es Client

export default async function Home() {
  const featured = await fetch(...)
  return (
    <div>
      <h1>Bienvenido</h1>
      <SearchBar />
    </div>
  )
}
```



Un Server Component **puede** importar Client Components. Un Client Component **no puede** importar Server Components directamente — pero puede recibirlos como children .

Rendering Strategies

SSG, SSR, ISR, CSR — cuándo usa cada uno

Las 4 estrategias

Estrategia	¿Cuándo renderiza?	Frescura	Performance	Caso típico
SSG	En build time	Congelada	⚡⚡⚡ Excelente	Blog, docs, landing
SSR	En cada request	Siempre fresca	⚡ Aceptable	Dashboard personalizado
ISR	Build + revalida cada N seg	Casi fresca	⚡⚡⚡ Excelente	News, productos e-commerce
CSR	En el browser	Siempre fresca	⚡ Depende del user	Apps muy interactivas



Next.js te deja **mezclar estrategias** en la misma app — cada ruta puede elegir la que mejor le sirva.

Next decide por ti (con defaults sensatos)

Por default, todo es estático. Next solo cambia a dynamic cuando detecta que es necesario.



Estático por default

Si tu página solo muestra datos "públicos", Next la pre-renderiza en `build`.

Resultado: HTML estático servido por CDN, carga sub-segundo.

Esto es lo que pasa con `/`, `/about`, `/pricing`, etc.



Se vuelve dynamic si...

Usás `cookies()` o `headers()`

Usás parámetros de búsqueda (`searchParams`)

Hacés `fetch()` sin opt-in al cache (default Next 15)

Exportás `dynamic = 'force-dynamic'`

Next 15: para que la página sea estática, los fetches necesitan `revalidate`, `tags` o `force-cache`.



Regla: **dejá que Next decida**. Solo forzá dynamic cuando realmente lo necesites.

ISR — lo mejor de ambos mundos

Con `revalidate` :

```
// app/posts/[id]/page.tsx
export default async function Post({ params }) {
  const { id } = await params
  const post = await fetch(
    `https://api.example.com/posts/${id}`,
    { next: { revalidate: 60 } }
  ).then(r => r.json())

  return <article>{post.title}</article>
}
```

Cada página se genera estática, pero se re-genera en background cuando pasan 60 segundos desde la última request.

🕒 ¿Cómo funciona?

1. Primera visita → genera HTML.
2. Sigüientes visitas (dentro de N seg) → sirve el cache.
3. Al pasar N seg, la siguiente visita aún ve cache, pero dispara regeneración.
4. A partir de ahí, nuevo cache.

✓ Casos ideales

Posts de blog (revalidate cada hora)
Productos de e-commerce (cada 5 min)
News, precios, rankings

💡 ISR = performance de SSG + frescura casi-SSR. Es probablemente el default correcto para contenido que cambia pero no en tiempo real.

Data Fetching

Server-side por defecto, client-side cuando se necesita

Server-side fetch: async component

```
// app/posts/page.tsx - Server Component
export default async function PostsPage() {
  const posts = await fetch(
    'https://jsonplaceholder.typicode.com/posts'
  ).then(r => r.json())

  return (
    <ul>
      {posts.map((p: Post) => (
        <li key={p.id}>
          <h3>{p.title}</h3>
          <p>{p.body}</p>
        </li>
      ))}
    </ul>
  )
}
```



Diferencia con React tradicional

Sin `useEffect`

Sin `useState` para loading/error

Sin waterfall cliente-servidor

Solo `async/await` directo.



¿Qué llega al cliente?

Solo HTML renderizado. Cero JavaScript de fetching. Cero spinners intermedios.

Loading state con loading.tsx

¿Y mientras el fetch del servidor ocurre? Next muestra automáticamente el `loading.tsx` de la carpeta.

```
// app/posts/loading.tsx
export default function Loading() {
  return (
    <div class="space-y-3">
      <div class="h-6 bg-gray-200 animate-pulse rounded">
      <div class="h-6 bg-gray-200 animate-pulse rounded">
      <div class="h-6 bg-gray-200 animate-pulse rounded">
    </div>
  )
}
```

```
// app/posts/page.tsx
export default async function Posts() {
  const posts = await fetch('...').then(r => r.json())
  return <PostList posts={posts} />
}
```

Cómo funciona

Next envuelve `page.tsx` en un `<Suspense>` con el `loading.tsx` como fallback.

El layout (navbar, footer) aparece inmediato, y el contenido se reemplaza cuando el fetch termina.

Cero configuración — basta con que el archivo exista en la carpeta.

Alternativas

`<Suspense>` manual para fallbacks más finos dentro de una página.

Skeletons > spinners — dan la forma de lo que viene.

Combinable con streaming (varias suspensiones en la misma página).



El layout persiste, el loading aparece en su segmento

Cache y revalidación (Next 15)

Next extiende `fetch()` con opciones de cache. En Next 15 el default cambió: ya no se cachea en runtime, hay que opt-in explícito.

Default — auto no cache

```
// Next 15: NO se cachea en runtime
fetch('https://api.example.com/posts')
```

Dev: fetch en cada request. Build: fetch una sola vez si la ruta es estática. Producción + ruta dynamic: fetch en cada request.

Time-based revalidation

```
fetch('https://api.example.com/posts', {
  next: { revalidate: 60 }
})
```

Opt-in al cache. Re-fetch cada 60 segundos. Ideal para datos que cambian pero no en tiempo real.

Tag-based revalidation

```
fetch('...', { next: { tags: ['posts'] } })
```

```
// Luego, desde un server action:
revalidateTag('posts')
```

También opt-in al cache. Lo invalidás a demanda tras mutaciones.

Force-cache (cache indefinido)

```
fetch('...', { cache: 'force-cache' })
```

Opt-in explícito al cache permanente — útil para datos que nunca cambian (ej. config, constants).



Cambio clave Next 14 → 15: antes `fetch()` **se cacheaba por default**. Ahora es al revés: **no se cachea** salvo que lo pidas con `revalidate`, `tags` o `force-cache`.

Client-side fetching: ¿cuándo?

Aunque el server fetching es el default, hay casos donde sí necesitás fetchear en el cliente.

✓ Casos donde Sí querés client-side

- 🔄 Polling — data que cambia cada segundos
- 🔍 Search típeo con debouncing
- ✍ Mutaciones con optimistic updates
- ∞ Infinite scroll / paginación dinámica
- 🔗 Dependent queries — X depende de Y

⚠ Problema: hacerlo a mano es verboso

Sin librería, necesitás:

- `useState` para data, loading, error
- `useEffect` para el fetch
- cleanup de requests cancelados
- cache manual entre componentes
- invalidación tras mutaciones
- retry, refetch on focus, etc.

Y eso solo lo básico. Vamos a ver por qué.

¿Por qué una librería de data fetching?

Los problemas reales que aparecen al manejar fetch "a mano" con `useState` + `useEffect`.

1. Race conditions

El usuario tipea "abc" → fetch A. Luego "abcd" → fetch B.
Si A tarda más y llega después de B, pisa la respuesta correcta.

La UI muestra resultados de "abc" aunque el input diga "abcd".

2. Request duplication

Dos componentes renderizan al mismo tiempo y ambos necesitan `/api/user`.

Resultado: dos requests idénticos a la red.

Multiplicá esto por toda tu app.

3. Stale data sin saberlo

El usuario navega a otra página y vuelve — ¿refetch? ¿usa cache?

Abre otra tab, edita algo, vuelve — la UI muestra datos viejos.

Decidir "cuándo refetch" manualmente es error-prone.

4. Loading/error boilerplate

Cada componente repite el mismo patrón: `useState` × 3 + `useEffect` + `try/catch`.

Ni hablar de `retry`, `timeouts`, cancelación.

80% del código es infraestructura, no lógica.

Lo que una librería te da gratis

SWR y TanStack Query resuelven estos problemas con convenciones.



Cache compartida

Múltiples componentes con la misma queryKey = UN solo request. Deduplicación automática.



Stale-while-revalidate

Muestra el cache viejo al instante, hace el refetch en background, actualiza cuando llega.



Cancelación de requests

Si el query key cambia o el componente se desmonta, el fetch antiguo se cancela — adiós race conditions.



Refetch on focus/reconnect

Al volver a la tab o recuperar red, refresca. El usuario siempre ve data fresca sin F5.



Retry + backoff

Request falla → reintento automático con delays crecientes. Sin código extra.



Invalidación explícita

Después de un mutation, invalidás queries relacionadas y la UI se actualiza sola.

SWR — minimalismo de Vercel

```
'use client'
import useSWR from 'swr'

const fetcher = (url: string) =>
  fetch(url).then(r => r.json())

export default function Users() {
  const { data, error, isLoading } = useSWR(
    '/api/users',
    fetcher
  )

  if (isLoading) return <p>Cargando...</p>
  if (error) return <p>Error</p>

  return (
    <ul>
      {data.map(u => <li key={u.id}><u.name></li>)}
    </ul>
  )
}
```

Características

Creado por Vercel (los de Next).

Filosofía: "stale-while-revalidate".

API minimalista — un solo hook principal.

Revalidación automática al volver a la tab.

Ideal para

Proyectos pequeños o lectura dominante.

Cuando querés algo simple y liviano.

Integración rápida con Next.

TanStack Query — el más completo

Antes "React Query". Es la librería de data fetching más usada en el ecosistema React — y lo que recomendamos para apps serias.

```
'use client'
import { useQuery } from '@tanstack/react-query'

export default function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: async () => {
      const r = await fetch('/api/users')
      return r.json()
    },
  })

  if (isLoading) return <p>Cargando...</p>
  if (error) return <p>Error</p>

  return (
    <ul>
      {data.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  )
}
```



Por qué es tan usado

- queryKey para cache granular
- useMutation para escrituras
- useInfiniteQuery para paginación
- Optimistic updates built-in
- Devtools visuales
- Retry, focus refetch, offline, etc.



En Next — dónde vive

Siempre dentro de Client Components ("use client").
Necesita el QueryClientProvider como wrapper,
típicamente en un archivo providers.tsx importado
desde el layout raíz.

TanStack Query — Mutations + Optimistic

```
'use client'
import { useMutation, useQueryClient }
  from '@tanstack/react-query'

export default function AddTask() {
  const qc = useQueryClient()

  const mutation = useMutation({
    mutationFn: (title: string) =>
      fetch('/api/tasks', {
        method: 'POST',
        body: JSON.stringify({ title }),
      }).then(r => r.json()),

    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ['tasks'] })
    },
  })

  return (
    <button onClick={() => mutation.mutate('Nueva tarea')}
      {mutation.isPending ? 'Creando...' : 'Crear'}
    </button>
  )
}
```

El flujo estándar

1. Disparas `mutation.mutate(data)`
2. TanStack llama al endpoint
3. `isPending` está `true` durante el fetch
4. En `onSuccess`, invalidás el cache
5. La query de `['tasks']` se re-fetchea automáticamente

Optimistic updates

Podés actualizar el cache antes de que el request termine.
Si falla, revertís. La UI se siente instantánea.

Optimistic updates en detalle

En vez de "mandá → espera → actualizá UI", hacés "actualizá UI → mandá → si falla, revertí". El usuario siente la app instantánea.

```
'use client'
import { useMutation, useQueryClient }
  from '@tanstack/react-query'

export default function LikeButton({ postId }) {
  const qc = useQueryClient()

  const mutation = useMutation({
    mutationFn: () =>
      fetch(`/api/posts/${postId}/like`, { method: 'POST' })

    // 1. Actualización optimista
    onMutate: async () => {
      await qc.cancelQueries({ queryKey: ['post', postId] })
      const previous = qc.getQueryData(['post', postId])

      qc.setQueryData(['post', postId], (old) => ({
        ...old,
        likes: old.likes + 1,
        likedByMe: true,
      }))
    }
  })
```

Los 3 hooks que usamos

`onMutate` — corre antes del fetch. Actualizás la cache manualmente y guardás el estado previo.

`onError` — si el fetch falla, revertís al estado previo.

`onSettled` — sí o sí al final: invalidás para que el server sea la fuente de verdad.

✓ Cuándo usarlo

Acciones que casi siempre tienen éxito (likes, favoritos, toggles).

Cuando la latencia del fetch arruina la experiencia.

⚠ Cuándo NO usarlo

Operaciones críticas donde fallar silenciosamente confunde: pagos, bookings, envío de mensajes importantes. Ahí preferís un loading explícito.

SWR vs TanStack — comparación

Dimensión	SWR	TanStack Query	Cuándo importa
Tamaño del bundle	~4 KB	~13 KB	Apps muy lite
Mutations	<code>mutate()</code> manual	<code>useMutation()</code> dedicado	Apps con escrituras
Devtools	Plugin externo	Oficiales, completas	Debugging serio
Cache granular	Por clave simple	<code>queryKey</code> arrays	Cache compartida entre queries
Infinite queries	<code>useSWRInfinite</code>	<code>useInfiniteQuery</code>	Feeds, paginación scroll
Offline, retry, focus	Básico	Granular, configurable	Apps robustas

Matriz de decisión

¿Server-side, SWR o TanStack? Depende del caso.

Escenario	Recomendación	Razón	Ejemplo
Página pública con datos lentos	Server (fetch)	Pre-render + cache + SEO	Blog, landing
Contenido que cambia cada hora	Server + revalidate	ISR = performance + frescura	News, productos
Dashboard con polling	TanStack Query	Refetch interval + cache	Stats, métricas live
Forms con mutaciones	TanStack Query o Server Actions	Invalidación + optimistic	Crear tareas, comentarios
Search autocomplete	SWR o TanStack	Debouncing + cache por query	Buscador de productos
Perfil del usuario logeado	Server (cookies)	Auth + dynamic automático	Settings, billing



Regla general: **server first**. Solo movés al cliente cuando hay interactividad que lo justifique.

Próxima clase

Server Actions · Route Handlers · Metadata · Optimización · Auth

Server Actions

Mutaciones desde forms sin endpoints

Route Handlers

APIs dentro de Next

Metadata & SEO

OG tags, favicon, títulos

Optimización

next/image, fonts, streaming

Middleware

Auth, redirects, i18n

Auth Libraries

Auth.js, Clerk, Supabase

¿Preguntas?

Clase 12 – Next.js: Fundamentos y Rendering



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

IIC2182 – Primer Semestre 2026